UCLCHEM

A Gas-Grain Chemical Code for Astrochemistry

Workshop Overview

Day	Morning (10:00–12:00)	Afternoon (13:00–15:30)
Monday	Installation & setup *(Gijs)*	**Overview & history** *(Gijs, Serena)*
Tuesday	Chemistry & ice *(Tobias)*	Collapse & post-processing / NEATH *(Felix)*
Wednesday	Warm-up physics & radiative treatment *(Tram)*	Shocks, heating, cooling, cosmic rays *(Kasia, Tram, Gijs)*
Thursday	Advanced modelling *(Marcus)*	Elective / requested topics
Friday	Free collaboration day	—

Each afternoon 15:30–17:00: Continued tutorial, open discussion & participant contributions.

Coffee breaks at: 10.45 and at 15:15

Today’s Programme

- **Morning** (10:00–12:00) — Installation & setup
- **Afternoon** (13:00–15:15) — UCLCHEM History, overview and a starter tutorial
- **Discussion** (15:30–17:00) — Open discussion & participant contributions

> **Note**
>
> During this session we will focus on a birds-eye-view of UCLCHEM. During the coming three days we will discuss everything in more detail.

M

Part 1 — Motivation

UCLCHEM

A Gas-Grain Chemical Code for Astrochemistry

Holdship et al. 2017, AJ 154, 38

&

UCLCHEM 4.0: A Modern Open Source Gas-Grain Framework for Astrochemical Modeling.

Vermariën et al. 2026, in prep.

What This Talk Covers

Tue–Thu sessions go deep on each component, guided by dedicated experts.

The Observer's Problem

Telescopes like **ALMA** and **JWST** detect molecular line emission:

- Maps of CO, $\mathrm{CH_3OH}$, HCN, complex organics…
- Line ratios and column densities encode **temperature**, **density**, **cosmic rays**, **history**,

But to extract physical conditions from observed abundances, you need a **forward model**:

- *Given these physical conditions and chemical network, what abundances do we predict?*
- A chemical model takes **physical conditions** + **chemical network** as input, producing **abundances over time** as output.

The Interstellar Medium: The Chemical Playground

Environment	Temperature (K)	Density (cm^{-3})	Key processes
(Collapsing) Dark cloud	10 K	$10 – 10^6$	Freeze-out, CR ionization, ice chemistry
Prestellar Core	100–300 K	$10^6 – 10^8$	Slow ice sublimation
shocks	10–4000 K	10^4–10^7	Sputtering, rapid sublimation
Your model	??? K	??? cm^{-3}	???

Chemistry is **time-dependent**: the abundance of CO today depends on the full history of that parcel of gas.

The Modelling Challenge

1. **Time-dependent**: abundances evolve over $10^4 - 10^7$ years and can consists of many different *stages*
2. **Multi-phase**: species exist in the gas *and* frozen on grain surfaces
3. **Large network**: 300+ species coupled by 3,000+ reactions
4. **Non-linear and stiff**: every abundance depends on every other and the physics; timescales span many orders of magnitude

It is a large system of differential equations, requiring careful consideration of which numerical solver is used.

What UCLCHEM Gives You

- Time-dependent abundance profiles for any species in the network
- For any (combination of) astrophysical environment (collapse, shock, Prestellar Core, …)
- Starting from user-defined initial conditions
- Output can be passed into SpectralRadex for line radiative transfer modelling.

Part 2 — Physical Environments

Physical Parameters: Setting the Stage

All parameters are passed as a Python **dictionary**. Unspecified parameters take sensible defaults for a (dark) cloud context.

Parameter	Description	Typical value
`initialDens`	Initial gas density (cm^{-3})	10^2– 10^6
`initialTemp`	Initial temperature (K)	10– 300
`finalTime`	Simulation duration (years)	10^4– 10^7
`radfield`	UV radiation field (Habing)	$1 - 10^5$
`zeta`	Cosmic ray ionization rate (ζ_{gal})	$1 - 10^5 \times 1.3 \times 10^{-17}$
`baseAv`	Visual extinction at cloud edge	2

The Six Physics Modules

The physics module you pick drives **density** and **temperature** evolution. The chemistry is solved along whatever trajectory you then define.

Module	Physical scenario
Cloud	Spherical cloud: static or simple freefall collapse
Collapse	Gravitational collapse with realistic density profiles
Prestellar Core	Protostellar envelope: temperature rises with time
C-Shock	MHD shock with ion-neutral drift
J-Shock	Jump shock with discontinuous front
Postprocessing	Post-processing of (hydrodynamical) simulations

Density and Temperature Evolution

Three illustrative scenarios:

- **Cloud** — static density and temperature
- **Collapse** — density rises as $\dot{n} \propto n^{4/3}$ (freefall), temperature stays cold (~10 K)
- **Prestellar Core** — after collapse, temperature rises as $T = 10 + A\,t^B (r/R)^{1/2}\ \mathrm{K}$, driven by protostellar luminosity (constants A, B set by stellar mass)
- **C-shock** — density and temperature spike over the dissipation length, then recover; grain ices are sputtered at the saturation time

> **Note**
>
> Each physics module is covered in depth by a dedicated session later this week.

1D Structure and UV Shielding

The cloud is divided into N equal segments. Default: **one point** (appropriate for UV-shielded, homogeneous regions).

Column density accumulated inward (more detail in Tram's talk on Wednesday):

$$N_{\mathrm{Tot}} = \frac{r_{\mathrm{cloud}}}{N_{\mathrm{points}}}\, n(r)$$

Visual extinction:

$$A_V = A_{V,0} + \frac{N_{\mathrm{Tot}}}{1.6 \times 10^{21}\ \mathrm{cm}^{-2}}$$

UV attenuation at each depth:

$$F_{\mathrm{UV}} = F_0 \exp(-k\, A_V)$$

Self-shielding of H_2, **C**, and **CO** is included. If you need a full PDR treatment, use 3D-PDR instead.

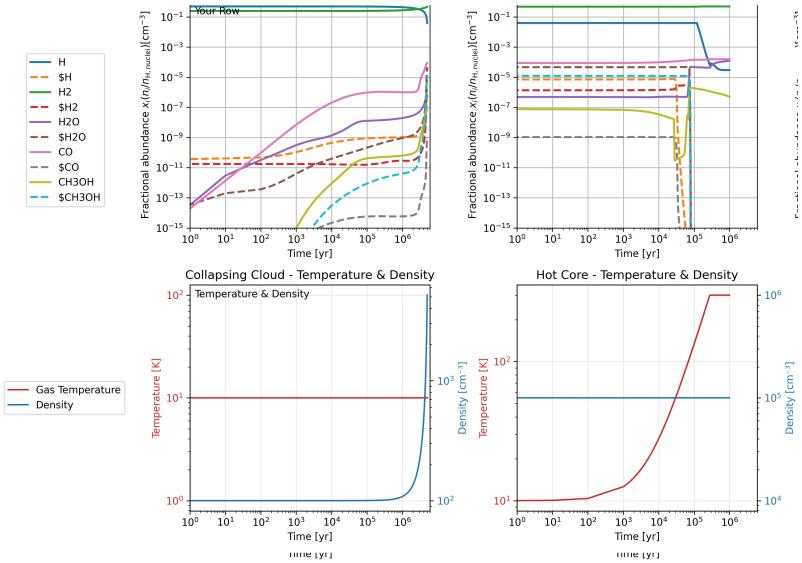
Multi-Stage Modelling

Chemistry has **memory**: the history of a cloud matters. Common examples include:

- Cold collapse → warm-up: ices build up during collapse, then sublimate in the warm-up
- Collapse → shock: the shock chemistry depends on the pre-shock abundances
- Post-processing: the chemistry depends on the full history of density and temperature in the simulation

The final abundances of one stage become the **initial abundances** of the next. You chain them like this:

```
cold = uclchem.model.Cloud(param_dict={...})
hot  = uclchem.model.HotCore(
    param_dict={...},
    previous_model=cold,
)
```



Part 3 — Gas-Phase Chemistry

The Chemical Network

Quantity	Value
Gas-phase reactions	2,000 (UMIST22 default)
Grain surface reactions	800+
Total species	300+
Grain species (surface + bulk)	~200

Species notation at a glance:

Notation	Meaning
`CO`, `H2O`	Gas phase
`#CO`, `#H2O`	Grain surface
`@CO`, `@H2O`	Bulk ice
`$CO`	Total ice = surface + bulk

Gas-Phase Reaction Types

Type	Example	Rate form
Photodissociation	`$\mathrm{CO + h\nu → C + O}$`	$\alpha F_{\mathrm{UV}} \exp(-kA_V)$
Cosmic ray ionization	`$\mathrm{H_2 + CRP → H_2^+ + e^-}$`	$\alpha\,\zeta$
CR-induced photons	secondary UV	$\alpha(T/300)^{\beta}\,\zeta$
Two-body collision	`$\mathrm{H + OH → H_2O}$`	$k\,X_i X_j n_H$
Ion-neutral	`$\mathrm{H_3^+ + CO → HCO^+ + H_2}$`	$k\,X_i X_j n_H$

Rate Coefficients: The Modified Arrhenius Equation

For the gas-phase reactions:

$$k = \alpha \left(\frac{T}{300\ \mathrm{K}} \right)^{\beta} \exp\left(-\frac{\gamma}{T} \right)$$

- α: pre-exponential factor
- β: temperature power-law index
- γ: activation barrier (K)

Other reaction parameterizations:

- UV: $k = \alpha\, F_{\mathrm{UV}}\ \exp(-k\, A_V)$
- Cosmic ray: $k = \alpha\, \zeta$ where ζ is in units of $1.3 \times 10^{-17}\ \mathrm{s}^{-1}$

Temperature ranges ($T_{\mathrm{min}}, T_{\mathrm{max}}$) from laboratory measurements; extrapolation is supported and recommended.

UMIST vs. KIDA

Both are well established gas phase chemical databases:

Database	Reference	Status
UMIST22	Millar et al 2024 (A&A 682, A109)	Default in UCLCHEM 4.0
KIDA24	Wakelam et al. 2024 (A&A 689, A63)	Supported but not well verified yet

The **same Fortran code** runs either network. The choice is made during the MakeRates and compilation step.

The ODE System for Gas Chemistry

For each species i, the fractional abundance $X_i = n_i / n_{\mathrm{H},nuclei}$ evolves as:

$$\frac{dX_i}{dt} = \sum_{j,k} k_{jk}\, X_j X_k\, n_{\mathrm{H}} + \sum_j k_j\, X_j - \sum_k k_{ik}\, X_k\, n_{\mathrm{H}}\, X_i - \sum_l k_l\, X_i$$

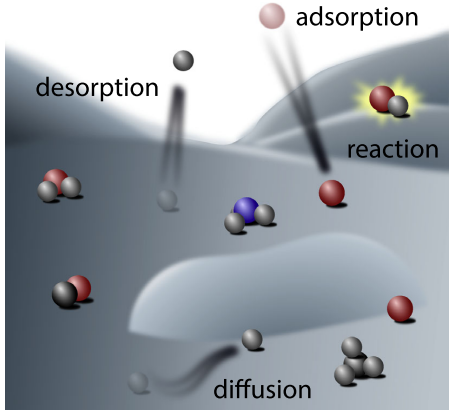
~300 species → **300 + 2 (dn, dT) coupled non-linear ODEs**

The system is **stiff**: some reactions happen on timescales of seconds (surface diffusion), others on timescales of millions of years (slow gas-phase reactions). Explicit integration methods would be impossibly slow.

Part 4 — Gas-Grain Chemistry

Dust Grains: Why They Matter

- At cold temperatures (< ~100 K): molecules collide with grains and **stick** onto the surface. This is freeze-out.
- The grain surface acts as a catalyst, enabling formation of more complex molecules on the surface.



Classic example — **methanol formation**:

$$\mathrm{CO(gas)} \xrightarrow{\text{freeze}} \backslash\#\mathrm{CO} \xrightarrow{+\backslash\#\mathrm{H}} \backslash\#\mathrm{HCO} \xrightarrow{+\backslash\#\mathrm{H}} \backslash\#\mathrm{H_2CO} \xrightarrow{+\backslash\#\mathrm{H}} \backslash\#\mathrm{CH_3O} \xrightarrow{+\backslash\#\mathrm{H}} \mathrm{CH_3OH(gas)}$$

Cuppen et al. 2017

Without grain surfaces, methanol formation in the ISM would be impossible.

Freeze-Out: Gas → Grain Surface

$$k_{\text{freeze}} = \underbrace{\left(1 + \frac{\beta \cdot 1.671 \times 10^{-3}}{r_g\, T_{\text{gas}}}\right)}_{\text{ion correction}} \times \alpha \times v_{\text{th}} \sqrt{\frac{T_{\text{gas}}}{m}} \times \frac{\sigma_g}{n_{\text{H}}}$$

where β is the ion charge (0 for neutrals), α the sticking coefficient, and σ_g / n_{H} the grain cross-section per H nucleus.

Switched off when $T_{\text{dust}} > T_{\text{max}}$ or desorption rate $>$ freeze-out rate.

Surface Reaction Mechanisms (More by Tobias Tomorrow)

- **Langmuir-Hinshelwood (LH)** — thermal diffusion: both species hop between binding sites until they meet. Stops above ~150 K. Example: `#H + #H → #$\mathrm{H}_2$`
- **Eley-Rideal (ER)** — direct impact: a gas-phase species lands directly on an adsorbed reactant. No diffusion required. Example: `H(gas) + #CO → #HCO`
- Both can produce **gas-phase** or **surface** products; branching ratio set by reaction exothermicity (Minissale et al. 2016)

Desorption: Surface to Gas

Four mechanisms return species to the gas phase:

Mechanism	Rate	Dominant where
Thermal	$k = \nu_0 \exp(-E_B/T)$	Warm regions, Prestellar Cores
Cosmic ray	CR impact heats grain transiently	Dense, UV-shielded clouds
Photodesorption	UV photon ejects surface molecule	PDR surfaces, cloud edges
Chemical	Exothermic surface reaction releases product	Everywhere

- Master switch: `desorb=True/False`
- Individual switches: `h2desorb`, `crdesorb`, `uvdesorb`

The Three-Phase Model

Reminder about notation:

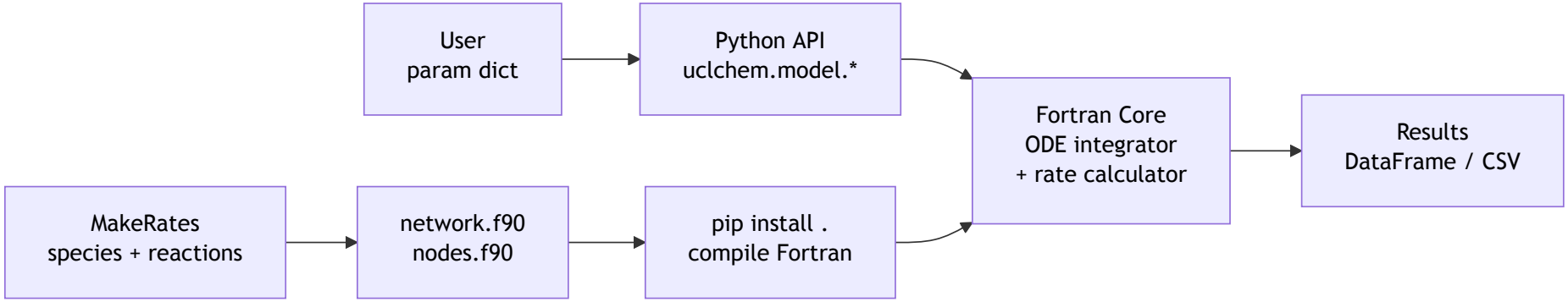
Phase	Notation	Description
Gas	`CO`	Free molecules
Surface	`#CO`	Adsorbed outer monolayer
Bulk ice	`@CO`	Buried in the ice mantle
Total ice	`$CO`	`#CO` + `@CO`

Surface $\leftrightarrow$ **bulk transfer** via:

- **Geometric**: as new ice layers grow, old surface becomes bulk; as ice erodes, bulk is exposed
- **Swapping**: individual molecules can swap between surface and bulk

Part 5 — The UCLCHEM Framework

Architecture: Python + Fortran



- **Python**: model classes, analysis tools, and parameter handling. All user-facing.
- **Fortran**: ODE integration and reaction rate calculation, with the network **hard-coded** for speed.
- Each UCLCHEM installation contains exactly **one chemical network**.
- It cannot be distributed as a pre-built wheel and must be compiled locally.

Explicit vs. Implicit Integration

Explicit (Forward) Euler — uses the current state to step forward:

$$X^{n+1} = X^n + \Delta t\, f(X^n)$$

Implicit (Backward) Euler — uses the next state:

$$X^{n+1} = X^n + \Delta t\, f(X^{n+1})$$

The implicit form requires solving a nonlinear system at each step — but that cost is worth it.

Stability consequences for stiff chemistry:

Method	Stability	Step size constraint
Explicit	Conditionally stable	$\Delta t < 2/\|\lambda_{\max}\|$
Implicit	Unconditionally stable	No constraint

The fastest eigenvalue in our network ($\lambda_{\max}$) comes from fast surface reactions. An explicit solver is forced into millisecond steps. An implicit solver can take steps of years.

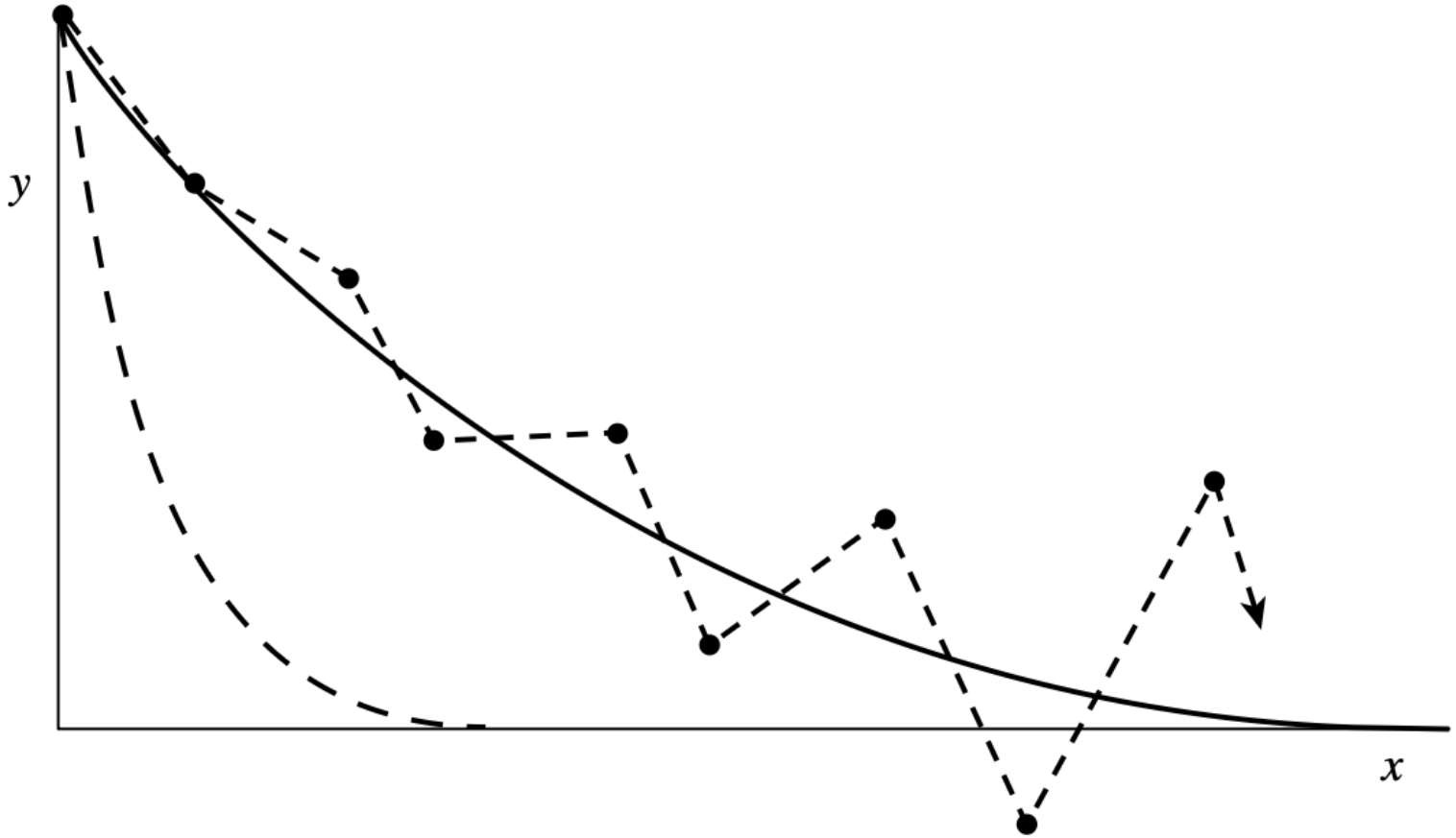


Figure 16.6.1. Example of an instability encountered in integrating a stiff equation (schematic). Here it is supposed that the equation has two solutions, shown as solid and dashed lines. Although the initial conditions are such as to give the solid solution, the stability of the integration (shown as the unstable dotted sequence of segments) is determined by the more rapidly varying dashed solution, even after that solution has effectively died away to zero. Implicit integration methods are the cure.

DVODE: The Solver Behind UCLCHEM

UCLCHEM uses **DVODE** (Brown, Byrne & Hindmarsh 1989), a workhorse of stiff chemistry solvers.

- Implements **Backward Differentiation Formulas (BDF)** — variable order 1–5
- **Variable stepsize**: adapts Δt to the current stiffness of the network
- **Variable order**: higher order = fewer steps, more memory per step. DVODE picks the most efficient combination.

Two tolerances control solution quality:

Parameter	Role	Default
`reltol`	Relative error per step	~10^{-8}
`abstol_factor`	Absolute error floor (for near-zero abundances)	~10^{-20}
`abstol_min`	Absolute error floor (for all abundances)	~10^{-20}

Error codes: `0` = success; negative values = failure.

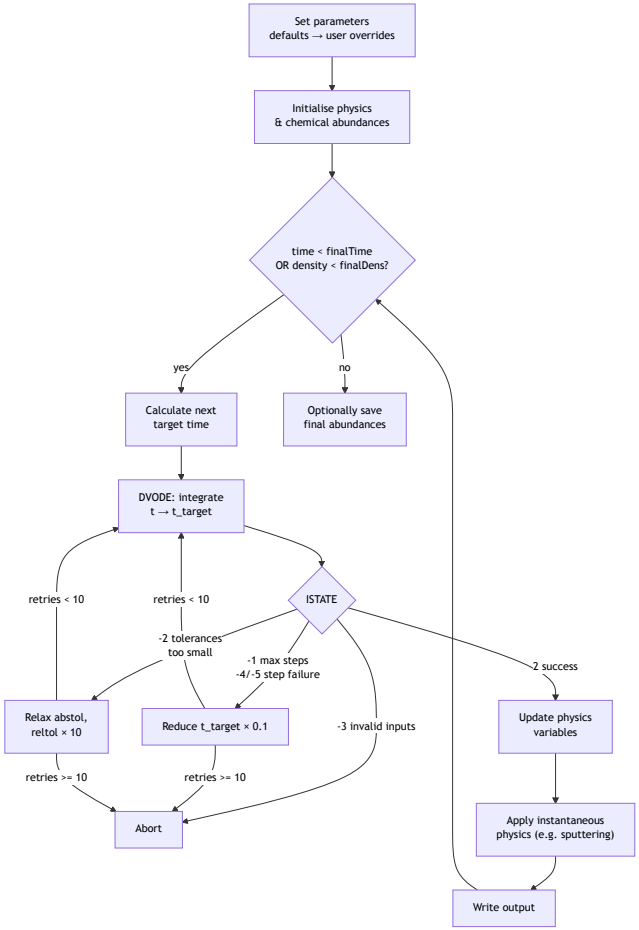
We always validate after running a model:

```
model.check_error()
model.check_conservation(element_list=["H", "C", "N", "O", "S"])
```

Important

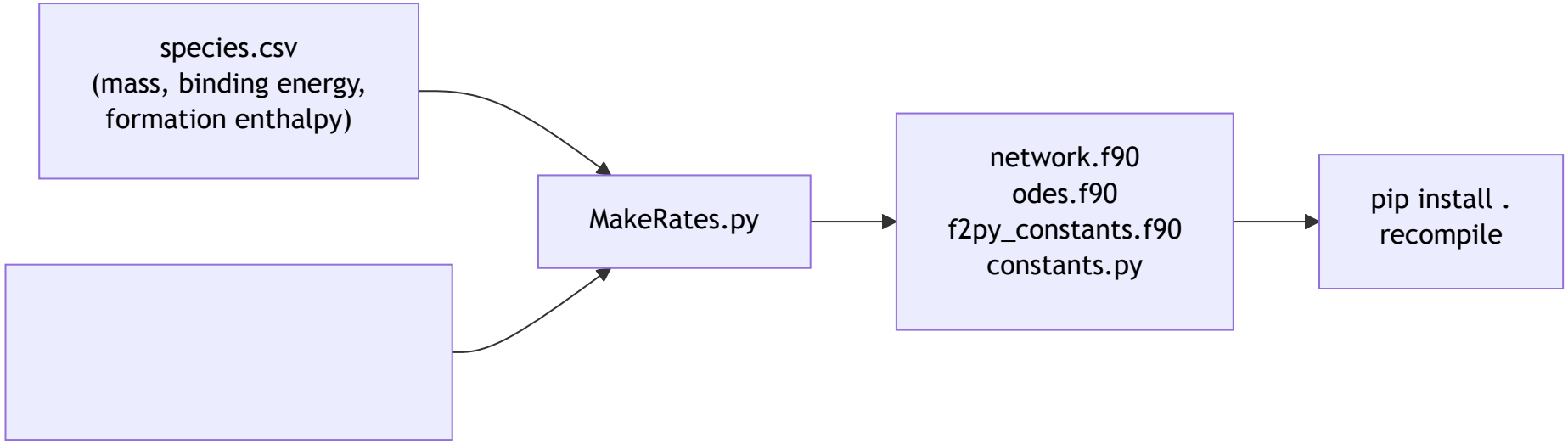
If element conservation is > 1%, tighten tolerances: `"reltol": 1e-12, "abstol_factor": 1e-20, "abstol_min": 1e-25`

The Main Integration Loop



MakeRates: Customising the Network

MakeRates is a **preprocessing tool** — generates the Fortran source compiled into UCLCHEM.



When to use it:

- Changing chemical network (e.g. UMIST → KIDA)
- Custom ice chemistry
- Adding new species or reactions

Workflow:

```
cd Makerates && python MakeRates.py    # edit user_settings.yaml first
cd .. && pip install .                 # always recompile after
```

> **Note**
>
> For this school we use the default UMIST22-based network. MakeRates is for advanced customisation.

Outputs and Analysis Tools

File outputs (legacy) (controlled via parameters):

Parameter	Content
`outputFile`	All species at all timesteps
`columnFile`	Selected species (`out_species`) only
`ratesFile`	Individual reaction rates
`abundSaveFile` / `abundLoadFile`	Persist abundances between stages

Python analysis API:

```
df  = model.get_dataframes()          # pandas DataFrames
fig = model.create_abundance_plot(species=["CO", "$CO"])
model.check_conservation(["H", "C", "N", "O", "S"])
```

Part 6 — Practical Preview

Running Your First Model

```python
import uclchem

params = {
    "initialDens": 1e4,   # cm^-3
    "initialTemp": 10.0,  # K
    "finalTime":   1e6,   # years
    "freefall":    False,
}

cloud = uclchem.model.Cloud(
    param_dict=params,
)

cloud.check_error()
cloud.check_conservation(element_list=["H", "C", "N", "O", "S"])

fig, ax = cloud.create_abundance_plot(
    species=["CO", "$CO", "H2O", "$H2O", "CH3OH", "$CH3OH"]
)
```

Multi-Stage Workflow Example

```python
# Stage 1: cold dark cloud collapse — build up ices
cold = uclchem.model.Cloud(
    param_dict={
        "initialDens": 1e4, "finalDens": 5e6,
        "freefall": True,   "finalTime": 3e4,
    }
)
cold.check_error()

# Stage 2: Prestellar Core warm-up — sublimate ices
hot = uclchem.model.PrestellarCore(
    param_dict={
        "initialDens": 1e6, "finalTime": 1e6,
        "baseAv": 20,
    },
    previous_model=cold,
)
hot.check_error()
```

The molecules that build up on the grains as the cold gas collapses are released back into the gas phase in the warm-up phase.

What’s Coming This Week

Day	Morning (10:00–12:00)	Afternoon (13:00–15:30)
Monday	Installation & setup *(Gijs)*	**Overview & history** *(Gijs, Serena)*
Tuesday	Chemistry & ice *(Tobias)*	Collapse & post-processing / NEATH *(Felix)*
Wednesday	Warm-up physics & radiative treatment *(Tram)*	Shocks, heating, cooling, cosmic rays *(Kasia, Tram, Gijs)*
Thursday	Advanced modelling *(Marcus)*	Elective / requested topics
Friday	Free collaboration day	—

Each afternoon 15:30–17:00: Continued tutorial, open discussion & participant contributions.

Coffee breaks at: 10.45 and at 15:15

Version: **v4.0.0-rc3** — full documentation at uclchem.github.io